



Bash Scripting Essentials

Introduction

Imagine you need to:

- **Automate repetitive tasks** like backups and updates.
- **Write a script to process data quickly.**
- **Customize system behavior with startup scripts.**

How do you achieve this efficiently in Linux?

This is where **Bash scripting** helps!

Section 1: Learn

What is Bash Scripting?

Bash scripting is a way to automate commands in Linux using a series of instructions written in a file. It helps:

1. **Automate repetitive tasks to save time.**
2. **Run multiple commands in a sequence.**
3. **Customize Linux environments and workflows.**

Why Do We Need Bash Scripting?

Without Bash Scripting	With Bash Scripting
Must type commands manually	Automates tasks
Hard to manage long commands	Runs complex operations easily



Without Bash Scripting	With Bash Scripting
Difficult to process large files	Handles large data efficiently
No customization of system tasks	Allows scheduling and automation

How Does Bash Scripting Work?

1. You write a sequence of commands in a file.
2. The system reads and executes the script line by line.
3. The output is displayed or logged as required.

An Interesting Anecdote

Bash (Bourne Again Shell) was created in **1989** as an improved version of the Unix shell. Today, **almost every Linux system** supports Bash scripting, making it a key tool for system administrators and developers!

Section 2: Practice

Step-by-Step Guide to Bash Scripting

1. Creating a Bash Script

```
# Create a new script file
```

```
nano myscript.sh
```

```
# Add the following lines:
```

```
#!/bin/bash
```

```
echo "Hello, World!"
```



Why is this useful?

- The `#!/bin/bash` line tells the system **to use Bash to execute the script**.
 - The `echo` command **prints text to the terminal**.
-

2. Making a Script Executable

```
# Give execution permission to the script  
chmod +x myscript.sh  
  
# Run the script  
./myscript.sh
```

What happens here?

- `chmod +x` makes the script executable.
 - `./myscript.sh` runs the script.
-

3. Using Variables in Bash

```
#!/bin/bash  
name="John"  
echo "Hello, $name!"
```

How does this help?

- Variables **store data** for use in the script.
 - `$name` retrieves the variable value.
-



4. Using Conditional Statements (**if**)

```
#!/bin/bash

echo "Enter a number:"
read number

if [ $number -gt 10 ]; then
    echo "Number is greater than 10"
else
    echo "Number is 10 or less"
fi
```

Why is this important?

- Adds **decision-making** capabilities to scripts.
 - Reads user input and **executes different actions** based on conditions.
-

5. Using Loops (**for** and **while**)

For Loop:

```
#!/bin/bash

for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

While Loop:

```
#!/bin/bash

count=1
```



```
while [ $count -le 5 ]; do
    echo "Count: $count"
    ((count++))
done
```

How does this help?

- Loops **repeat tasks multiple times**, reducing manual effort.
-

6. Creating Functions in Bash

```
#!/bin/bash

greet() {
    echo "Hello, $1!"
}

greet "Alice"
greet "Bob"
```

Why is this useful?

- Functions **reuse code efficiently**.
 - **\$1** represents **the first argument passed to the function**.
-

7. Scheduling a Bash Script with Crontab

```
# Open crontab
crontab -e

# Schedule a script to run daily at 2 AM
```



```
0 2 * * * /home/user/myscript.sh
```

How does this help?

- Runs scripts **automatically at scheduled times.**
-

Real-World Example

A system administrator needs to:

1. Backup important files every night.
2. Delete old log files automatically.
3. Send daily status emails to the IT team.

What did they do?

1. Wrote a script to **backup files using tar.**
2. Used **find /var/log -mtime +7 -delete** in a script to delete old logs.
3. Scheduled the script with **crontab** to run every night at 1 AM.

Result? The entire process was fully automated!

Section 3: Know More

Frequently Asked Questions

1. How do I pass arguments to a script?

```
./myscript.sh argument1 argument2
```

2. How do I debug a Bash script?

```
bash -x myscript.sh
```

3. How do I handle user input in Bash?



```
read name  
echo "Hello, $name!"
```

4. How do I log script output to a file?

```
./myscript.sh > output.log 2>&1
```

5. Where can I practice Bash scripting?

- Use a Linux Virtual Machine.
- Try online Bash simulators like JSLinux.

Conclusion

Bash scripting is an essential skill for automating tasks, managing servers, and improving efficiency.

Start by writing simple scripts, using variables, and adding logic, and soon you'll be automating Linux like a pro!